Информация о площадках программы

Подробнее

до 11 янв., 01:30

Открыть урок

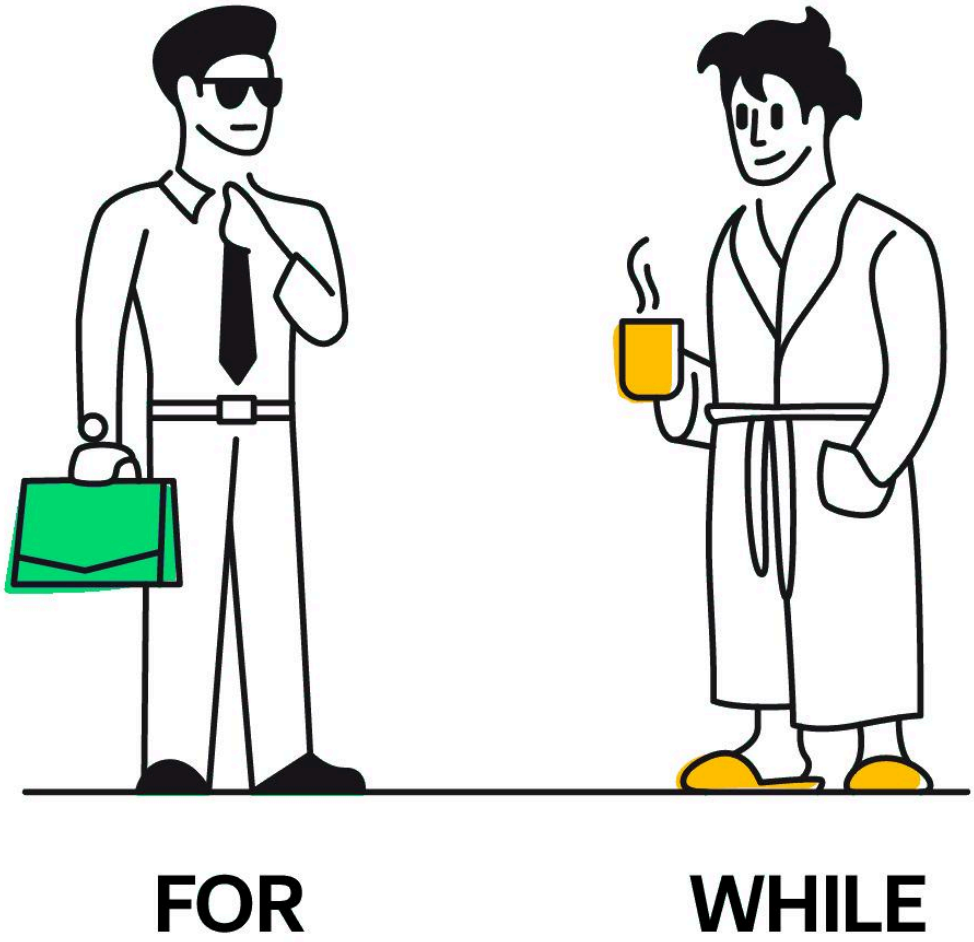
Когда: 24.10.26 в 16:00

Оператор for. Не будем зацикливаться на while

- Цикл for по элементам
- Цикл for по диапазону
- Аргументы функции print
- Кодирование символов

На прошлом уроке про `while` был спойлер, что операторов цикла в Python целых два. И пришло время сбросить завесу тайны с этого вопроса.

Представьте, что вы решили много раз написать одну и ту же фразу. Неважно зачем — может, вы пообещали себе сто раз повторить аффирмацию «я выучу Python» (но лучше бы задачи решали). Как бы то ни было, делать одно и то же действие много раз подряд вручную — занятие скучное и бесполезное. К счастью, в Python есть специальный оператор цикла с известным числом повторений — `for`.



✅ Нет, ну это перебор

Самые внимательные из вас (❤️) уже справедливо заметили, что задачу повторить какое-то действие *n* раз можно «на изи» решить с помощью цикла `while` со счётчиком:

```
counter = 0
while counter < 100:
    print("Я выучу Python")
    counter += 1
```

С этим утверждением (и кодом выше) спорить бесполезно. Откроем секрет: любой алгоритм, написанный с циклом `while`, можно переписать на `for` и наоборот. Важно, насколько простым и читаемым получится результат.

Вообще, задача цикла `for` в Python несколько шире, чем в некоторых других языках программирования: **берём каждый элемент из последовательности и выполняем блок кода для этого элемента**.

То есть `for` **перебирает последовательности**. Общий синтаксис выглядит так:

```
for переменная in последовательность:
    # Делаем что-то с переменной.
```

Проблема в том, что с последовательностями мы пока не очень знакомы (пусть и скоро познакомимся). Поэтому давайте поэкспериментируем.

```
for num in 1, 2, 3, 4, 5:  # Выглядит как последовательность.
    print(num)  # Работает! Ну не чудо ли?
```

Что происходит в коде выше? На первой итерации цикла переменная `num` принимает значение 1, затем 2 и так до конца.

```
for line in input(), input(), input():
    print('Пользователь ввёл строчку:')
    print(line)
```

Обратите внимание: **до того, как начать выполнять тело цикла, интерпретатор сначала вычислил все функции в заголовке** — заставил пользователя ввести три строки. То есть фактически код выше равнозначен этому:

```
line1 = input()
line2 = input()
line3 = input()
for line in line1, line2, line3:
    print('Пользователь ввёл строчку:')
    print(line)
```

Итак, чтобы у нас было, что перебирать, можно в заголовке цикла просто перечислить то, что нам хочется, и тогда переменная цикла будет последовательно принимать заданные нами значения. А когда значения закончатся, то мы просто выйдем из цикла.

Продолжим эксперименты. Помните, что применительно к строкам мы несколько раз употребляли слово **«коллекция»** и даже искали в строке максимальную и минимальную букву? Может, можно перебирать буквы в строке?

```
for fruit in "🍎🍌🍊":
    print(f"Съел {fruit}.")  # Работает!
print("Фрукты закончились. Теперь чипсы?")
```

Ещё пример:

```
print('То, что вы скажете, будет использовано против вас побуквенно!')
line = input()
for letter in line:
    print(letter)
print('Как мы и предупреждали.')
```

Более того, в заголовке цикла `for` работает и множественное присваивание. Пока посмотрим на следующий пример, как на волшебство, а уже скоро подробнее выясним, как оно там работает под капотом:

```
for l1, l2 in 'он', 'ты', 'мы':
    print(f'В двухбуквенном слове {l1}{l2}')
    print(f'Первая буква {l1}, вторая - {l2}')
```

📈 Будем последовательны

А то мы слишком разогнались.

Давайте вернёмся к примеру с повторением строки 100 раз. Испугались, что хотим заставить вас перечислять в заголовке цикла числа до ста? Формально так и нужно сделать, но фактически в Python для решения такой задачи есть возможность создать последовательность целых чисел с помощью `range()`:

```
for _ in range(100):
    print("Я выучу Python")
```

Мы говорили (и продолжаем говорить), что надо давать осмысленные имена переменным. Но, по соглашению PEP8, переменные цикла, которые используются как счётчики, можно называть одной буквой — `i`, `j`, `k`. Можно вообще использовать для имени нижнее подчёркивание `_`, но только в том случае, если мы хотим сделать акцент: внутри цикла переменная никак не используется и нужна лишь для организации нужного количества повторений.

Получилось в два раза короче, чем с циклом `while`, и не надо менять счётчик самому.

Давайте внимательнее посмотрим, какую последовательность создаёт `range()`.

```
for i in range(5):
    print(f"День {i}: Я всё ещё не умею летать 🐦")
```

Цикл выполнился ровно пять раз, но `i` принимала значения 0, 1, 2, 3, 4. Это связано с тем, что в программировании принято начинать счёт с нуля. Именно поэтому `range()` выдаёт значения с нуля до верхней границы диапазона не включительно.

Хм, а что, если мы пишем программы не для других программистов, а для нормальных людей, которые считают с единицы? Есть два способа решения проблемы:

- Увеличивать значение на единицу непосредственно перед выводом:

```
print("Посчитаю от единицы до введённого числа (включительно)")
for i in range(int(input())):
    print(i + 1)
```

- Вызвать `range()` двумя параметрами — началом и концом диапазона:

```
print("Посчитаю от единицы до введённого числа (включительно)")
for i in range(1, int(input()) + 1):  # Прибавили единицу, чтобы вошло са
    print(i)
```

Значит, `range()` умеет принимать настройку не только конца диапазона (не включительно), но и начала (включительно). А ещё он может выдавать числа не подряд, а с некоторым шагом. Допустим, нам надо вывести все чётные числа от одного до введённого включительно. Мы можем написать `if` внутри цикла:

```
for i in range(1, int(input()) + 1):
    if i % 2 == 0:  # Перебираем все и ищем подходящее.
        print(f'{i} - чётное')
```

А можем задать шаг в качестве третьего параметра `range()`:

```
for i in range(2, int(input()) + 1, 2):  # Обратите внимание, что идём с
    print(f'{i} - чётное')  # Выводим только нужное.
```

Подытожим. `range()` можно вызывать с разным количеством аргументов:

- С одним. `range(stop)` создаст последовательность чисел от нуля до `stop` не включительно.
- С двумя. `range(start, stop)` — последовательность от `start` включительно до `stop` не включительно.
- С тремя. `range(start, stop, step)` — последовательность от `start` включительно до `stop` не включительно с шагом `step`.

Но и это ещё не все. Никто не запрещает нам сделать шаг отрицательным. Тогда, чтобы цикл запустился хоть раз, надо задать `start` большим или равным `stop`:

```
# Считаем от 10 до 0 (не включительно) с шагом -1.
for countdown in range(10, 0, -1):
    print(f"🚀 {countdown}...")
print("Ту зе мун!")
```

Когда мы говорим про `range()`, то специально избегаем фразы «генерирует последовательность», потому что часто `range()` называют генератором, а это не так. На самом деле это (сейчас будет страшно) — **ленивая неизменяемая последовательность, для которой можно создавать итераторы**. (А-а-а-а-а-а! 😱) Когда мы доберёмся до подробностей, эта фраза уже не будет звучать так пугающе, обещаем. А пока важно знать, что `range()` не создаёт всю последовательность чисел сразу, а позволяет получать по одному числу за раз. Поэтому `range(1_000_000_000_000_000)` не съест всю оперативную память вашего устройства.

Ещё несколько трюков с `range()`:

```
my_range = range(1, 100, 2)  # Можно сохранить в переменную.

for i in my_range:
    print(i)

for j in my_range:  # Начнём сначала.
    print(j)
```

```
print(my_range.start,  # Если вдруг забыли, можно узнать начало,
      my_range.stop,  # конец,
      my_range.step)  # и шаг.

print(2 in my_range)  # Узнать, входит ли какое-то значение в последовате

print(len(my_range))  # Узнать длину последовательности.
```

Тюнингуем print

С использованием спецсимволов перевода строки и табуляции, а также после знакомства с f-строками вывод наших программ и так стал невероятно прекрасен. Но что, если `print()` тоже скрывает от нас ещё несколько возможностей?

Функция `print()` в Python очень гибкая. У неё есть именованные аргументы (*keyword arguments*), позволяющие настроить разделитель между выводимыми частями и тем, что будет в конце вывода:

- `sep` (separator) — строка-разделитель, которая вставляется между аргументами при печати. По умолчанию `sep=" "` (пробел). Можно ставить любой разделитель: запятую, тире, смайл, секретное послание на 10 листах — что угодно, лишь бы это была строка.

```
print("Cats", "Dogs", "Pythons", sep=" vs ")
# Вывод: Cats vs Dogs vs Pythons
```

- `end` — строка, которая добавляется в конце вывода. По умолчанию `end="\n"`, то есть каждый `print()` заканчивается переводом строки. Сделаем так, чтобы два вызова `print` напечатали в одной строке:

```
print("Привет", end=", ")
print("пользователь", end="!\n")
# Вывод: Привет, пользователь! + перевод на следующую строку.
```

PEP8

Вокруг знака = при указании значений именованных аргументов **пробелы не нужны**.

Комбинируя `sep` и `end`, и применяя знания, которые у нас уже есть, можно творить чудеса форматирования вывода.

Следующий лайфхак, строго говоря, вообще не относится к функции `print()` — оператор распаковки `*`. Это брат-близнец умножения, который помогает сделать много всякого. Но пока мы будем использовать его могущество для того, чтобы `print()` воспринимал строки и `range()` не как единое целое, а как набор входящих внутрь букв или цифр.

```
sample_range = range(10)
print(sample_range)  # range(0, 10)
print(*sample_range)  # 0 1 2 3 4 5 6 7 8 9

sample_string = "превысокомногорассмотрительствующий"
print(sample_string)  # превысокомногорассмотрительствующий
print(*sample_string)  # п р е в ы с о к о м н о г о р а с с м о т р и т
```

А раз `print()` начинает воспринимать что-то не как одно значение, а как несколько, значит, с помощью параметра `sep` можно влиять на разделители между элементами.

```
print(*'Повторяю по буквам', sep='-')  # П-о-в-т-о-р-я-ю- -п-о- -б-у-к-в-
print(*range(1, 101, 10), sep=", потом ")
# 1, потом 11, потом 21, потом 31, потом 41, потом 51, потом 61, потом 71
```

🐾 Почему цифровые котики откликаются на последовательность 1050, 1080, 1089, 45, 1082, 1080, 1089, 33

Картина: вы пытаетесь объяснить компьютеру, что такое буква А. Он смотрит на вас пустым экраном и спрашивает: «А — это 65? Или 1041? Или вообще 🍕?» Вы понимаете: чтобы общаться, нужен общий язык кодирования.

Любой текст в памяти компьютера — это числа. А кодирование — это словарь перевода между буквами и числами.

В 1960-х годах инженеры думали: «Хватит ли 128 символов для всего мира?» Ответ: «Конечно! У нас же есть латиница, цифры и символы вроде @ и #». Так появилась ASCII — таблица, где каждому символу присваивался семибитный код (0-127).

Пример:

```
A → 65
a → 97
! → 33
```

Через какое-то время появилась расширенная ASCII: 256 символов — уже прогресс! Теперь можно писать на кириллице, добавлять псевдографику (▄, ▌, ▐). Проблема была в том, что в такую кодировку можно было бы засунуть ещё только один язык, кроме английского, а японскому или китайскому места и вовсе не хватало.

Началась «Война кодировок» (KOI-8, Windows-1251, ISO-8859-5). И если вы не угадали — получите кракозябры вместо текста.

Наконец, появилась идея создать единую таблицу для всех символов всех языков, включая эмодзи и древние руны. Так появился `Unicode`.

Фишки Юникода:

- Каждому символу — уникальный код (например, А → U+0041, Б → U+0411, ☃ → U+2603).
- Поддерживает даже символы из вымышленных вселенных (например, клингонский).
- Коды не ограничены — таблица расширяется (сейчас > 150 000 символов).

Пример:

```
print('\U0001F4A9')  # 💩
```

Факт: **В Python3 все строки — юникодные** (тип str). Это значит:

```
text = 'Привет, 世界! 🌍'  # Работает из коробки.
```

Иногда бывает нужно преобразовать символ в его код или наоборот, получить из кода символ, который ему соответствует.

- Функция `ord()` превращает символ в его юникод:

```
print(ord('A'))    # 65
print(ord('Б'))    # 1041
print(ord('☃'))    # 9731 (это 0x2603 в HEX)
```

- Функция `chr()` делает обратное — превращает код в символ:

```
print(chr(65))     # A
print(chr(1041))   # Б
print(chr(9731))   # ☃
```

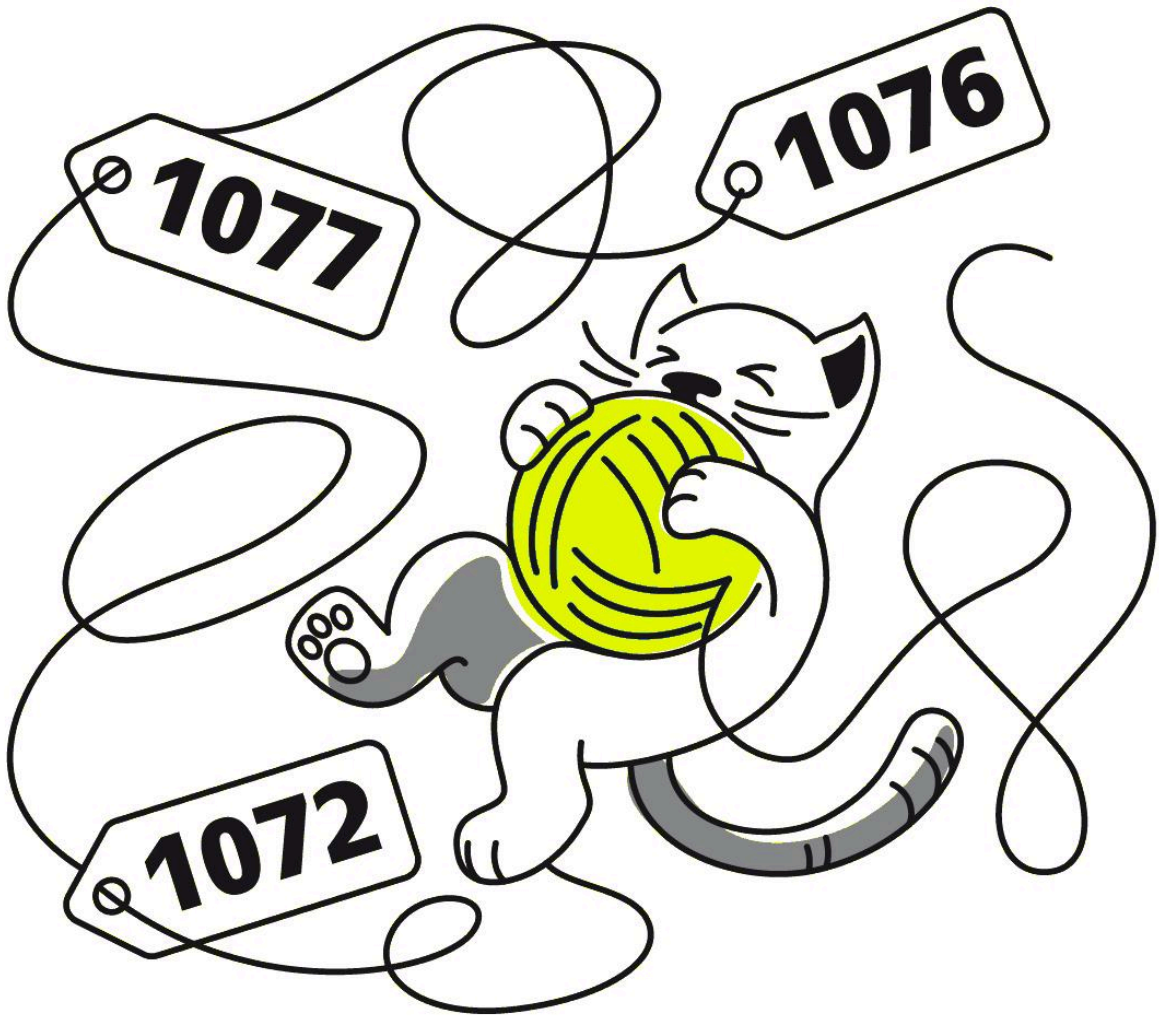
Вот так можно вывести заглавный английский алфавит:

```
for i in range(26):
    print(chr(ord('A') + i), end=' ')
```

С русским алфавитом такое тоже работает, но есть особенности у букв Ё и ё.

А так можно расшифровать послание из названия пункта учебника:

```
for i in 1050, 1080, 1089, 45, 1082, 1080, 1089, 33:
    print(chr(i), end='')
```



С помощью преобразований в код символа и обратно решаются задачи на реализацию шифров замены, например шифра Цезаря: при шифровании каждая буква исходного сообщения сдвигается на сколько-то позиций в одну сторону, а при расшифровке смещение происходит в обратную сторону:

```
secret = ''
shift = int(input())
for char in 'HELLO':
    secret += chr(ord(char) + shift)
print(secret)  # Расшифровка: смещение назад на -shift.
```

В программе выше есть ошибка, которая возникает при выходе за границы кодов латинского алфавита. Всё будет работать, но при достаточно большом смещении может произойти замена латинских букв на, например, китайские иероглифы, что

Цезаря бы точно смутило. Подумайте, как сделать эту программу лучше с использованием получения остатка от целочисленного деления.

🌟 Итог: прошли по всем пунктам

Цикл `for` — это прекрасный помощник по автоматизации, который не пропустит ни одного элемента в списке дел (если мы его дополнительно не попросим, но об этом на следующем уроке).

А пока мы научились:

- создавать разные последовательности целых чисел с помощью `range()`;
- перебирать числа, буквы и слова;
- совершили путешествие от букв к их кодам и обратно;
- поманили цифрового котика.

P. S. Если вы дочитали до конца — поздравляем, у вас успешно отработал цикл `for` по строкам этого урока.